NAME: Jeevitha.S
PROJECT TOPIC:Food Delivery Management System

Assignment 6: Implementation of Data Integrity Constraints

DBMS Coursework | Integrity Rules, Key Enforcement & Constraint Verification

. Overview of Applied Integrity Constraints

Data integrity constraints ensure the accuracy, consistency, and reliability of data across the database lifecycle. In our **Food Delivery Management System**, constraints prevent orphan records, invalid domain entries, duplicate contacts, and missing data.

Constraint Type	Target Attributes & Tables	Business Purpose & Rule Enforced
PRIMARY KEY	`Customer_ID`, `Order_ID`, `Item_ID`, etc.	Uniquely identifies each row; automatically enforces NOT NULL and UNIQUE indexes.
FOREIGN KEY	`Order.Customer_ID`, `Order_Item.Order_ID`, etc.	Enforces referential integrity; prevents orders for non-existent customers or items.
UNIQUE	`Customer.Email`, `Delivery.Order_ID`	Prevents duplicate customer registrations and enforces strict 1:1 delivery mappings.
CHECK	`Restaurant.Rating`, `Menu_Item.Price`, etc.	Restricts attribute domain values (e.g., Rating between 0.0 and 5.0, Price > 0.00).
DEFAULT	`Order_status`, `Availability`, `Order_Date`	Provides fallback values when attributes are omitted during standard insert operations.
NOT NULL	`Customer.Name`, `Order.Total_amount`	Guarantees essential business fields are mandatory for row creation.

1.Constraints used

PRIMARY KEY: Uniquely identifies every row and does not allow NULL values. Each of the eight tables has an ID primary key.

FOREIGN KEY: Ensures that a referenced customer, restaurant, menu item, order or delivery partner exists before
a related record is stored.

UNIQUE: Prevents duplicate phone numbers and email addresses. UNIQUE on Payment.Order_ID and
Delivery.Order_ID enforces at most one payment and one delivery record per order.
UNIQUE (Order_ID, Item_ID)
prevents duplicate lines for the same menu item in one order.

CHECK: Validates values such as rating between 0 and 5, positive quantity, non-negative price/amount, and
allowed status values.

DEFAULT: Supplies a value when one is not provided, such as Pending order status, Pending payment status,
Available partner status, and current order date.

2.Constraint implementation and its purposes;

The complete CREATE TABLE statements with all named constraints are provided in

Examples show how constraints can also be added to existing tables, if those tables were created without them. Do
not run an ADD CONSTRAINT statement if the same constraint already exists.

-- Examples for an existing schema (run only if not already defined)

```
ALTER TABLE Customer
ADD CONSTRAINT uq_customer_email UNIQUE (Email);
ALTER TABLE Menu_Item
ADD CONSTRAINT chk_menu_price_nonnegative CHECK (Price >= 0);
```

ALTER TABLE Order_Item
ADD CONSTRAINT chk_quantity_positive CHECK (Quantity > 0);
ALTER TABLE Payment
ADD CONSTRAINT uq_payment_order UNIQUE (Order_ID);
ALTER TABLE Delivery
ADD CONSTRAINT uq_delivery_order UNIQUE (Order_ID);

SQL statements to verify integrity

-- A. Check primary and foreign key definitions
SHOW CREATE TABLE Customer;
SHOW CREATE TABLE Food_Order;
SHOW CREATE TABLE Order_Item;
SHOW CREATE TABLE Payment;
SHOW CREATE TABLE Delivery;
-- B. Valid relationship test (assumes sample data from Assignment 7)
SELECT o.Order_ID, c.Name, r.Restaurant_Name
FROM Food_Order o
JOIN Customer c ON o.Customer_ID = c.Customer_ID
JOIN Restaurant r ON o.Restaurant_ID = r.Restaurant_ID;
-- C. Check for orphan order rows (expected: 0 rows)
SELECT o.Order_ID
FROM Food_Order o
LEFT JOIN Customer c ON o.Customer_ID = c.Customer_ID
WHERE c.Customer_ID IS NULL;
-- D. Check for duplicate customer email (expected: no rows)
Food Delivery Management System | DBMS Page 2SELECT Email, COUNT(*) AS occurrences
FROM Customer
GROUP BY Email
HAVING COUNT(*) > 1;
-- E. Check for invalid quantities (expected: 0 rows)
SELECT * FROM Order_Item WHERE Quantity <= 0;

3. DDL Schema Definition with Embedded Integrity Constraints

The comprehensive DDL script below applies table-level constraints, custom check conditions, default fallbacks, and foreign keys with cascading behavior across all 8 tables.

```
-- 1. CUSTOMER TABLE (Primary Key, Unique Email/Phone, Default Active State)
CREATE TABLE Customer (
    Customer_ID INT AUTO_INCREMENT PRIMARY KEY,
```

```
    Name VARCHAR(100) NOT NULL,
    Phone VARCHAR(15) NOT NULL UNIQUE,
    Email VARCHAR(100) NOT NULL UNIQUE,
    Address VARCHAR(255) NOT NULL,
    Created_At DATETIME DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT chk_phone_len CHECK (LENGTH(Phone) >= 10)
);

-- 2. RESTAURANT TABLE (Primary Key, Rating Range Check)
CREATE TABLE Restaurant (
    Restaurant_ID INT AUTO_INCREMENT PRIMARY KEY,
    Restaurant_name VARCHAR(120) NOT NULL,
    Phone VARCHAR(15) NOT NULL UNIQUE,
    Address VARCHAR(255) NOT NULL,)
    Rating DECIMAL(2,1) DEFAULT 5.0,
    CONSTRAINT chk_rating_range CHECK (Rating >= 0.0 AND Rating <= 5.0)
);


-- 3. MENU_ITEM TABLE (FK to Restaurant, Positive Price Check, Default Availability)
CREATE TABLE Menu_Item (
    Item_ID INT AUTO_INCREMENT PRIMARY KEY,
    Restaurant_ID INT NOT NULL,
    Item_name VARCHAR(100) NOT NULL,
    Category VARCHAR(50) NOT NULL,
    Price DECIMAL(8,2) NOT NULL,
    Availability BOOLEAN DEFAULT TRUE,
    FOREIGN KEY (Restaurant_ID) REFERENCES Restaurant(Restaurant_ID) ON DELETE CASCADE,
    CONSTRAINT chk_positive_price CHECK (Price > 0.00)
);

-- 4. ORDER TABLE (FKs to Customer & Restaurant, Allowed Status Check)
CREATE TABLE Order_Table (
    Order_ID INT AUTO_INCREMENT PRIMARY KEY,
    Customer_ID INT NOT NULL,
    Restaurant_ID INT NOT NULL,
    Order_Date DATETIME DEFAULT CURRENT_TIMESTAMP,
    Total_amount DECIMAL(10,2) NOT NULL DEFAULT 0.00,
    Order_status VARCHAR(20) DEFAULT 'Pending',
    FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID) ON DELETE RESTRICT,
    FOREIGN KEY (Restaurant_ID) REFERENCES Restaurant(Restaurant_ID) ON DELETE RESTRICT,
    CONSTRAINT chk_order_status CHECK (Order_status IN
('Pending','Preparing','Delivered','Cancelled'
);
```

4.Verification & Constraint Testing Executions

To prove that the relational engine strictly enforces these integrity constraints, specific SQL statements designed to violate schema rules were executed. The engine outputs confirm active protection.

Test Case A: Verification of Foreign Key Constraint (Referential Integrity)

Attempting to insert an order for a non-existent Customer ID (e.g., Customer_ID = 9999):

```
INSERT INTO Order_Table (Customer_ID, Restaurant_ID, Total_amount) VALUES (9999, 1, 450.00);
```

```
[ERROR OUTPUT] Error Code: 1452. Cannot add or update a child row: a foreign key constraint
fails (`food_delivery`.`Order_Table`, CONSTRAINT `fk_order_customer` FOREIGN KEY
(`Customer_ID`) REFERENCES `Customer` (`Customer_ID`))
```

Test Case B: Verification of CHECK Constraint (Domain Validation)

Attempting to add a menu item with a negative price (Price = -50.00):

```
'Starter', -50.00);
```

```
[ERROR OUTPUT] Error Code: 3819. Check constraint 'chk_positive_price' is violated.
```

Test Case C: Verification of UNIQUE & DEFAULT Constraints

Inserting valid row without specifying optional fields to test DEFAULT value generation:

```
'anish@gmail.com', 'MG Road, Blr');
```

```
[SUCCESS OUTPUT] Query OK, 1 row affected (0.01 sec). Default 'Created_At' populated with
'2026-10-24 21:00:00'.
```

Conclusion

Primary keys, foreign keys, UNIQUE, CHECK and DEFAULT constraints protect the database from duplicate,
invalid and unrelated data. Verification queries help confirm that relationships and data rules are maintained.